# Network Function Virtualization (NFV)

SPE Major Project

Complete Documentation & Setup Guide

**Diksha**
GitHub: `dikshax86`
DockerHub: `dknights`

May 16, 2026

# Contents

# 1 Project Overview

This project simulates **virtualized network functions** — a firewall, a switch, and a traffic monitor — deployed as microservices with a full CI/CD pipeline, container orchestration, monitoring, and centralized logging.

## 1.1 What is NFV?

Network Function Virtualization (NFV) is the concept of replacing dedicated network hardware (physical firewalls, switches, load balancers) with software running on standard servers. In this project, we implement three core network functions as lightweight Python microservices:

- **Switch** — Routes network traffic to appropriate destinations
- **Firewall** — Inspects and filters traffic based on rules
- **Monitor** — Logs all traffic for auditing and analysis

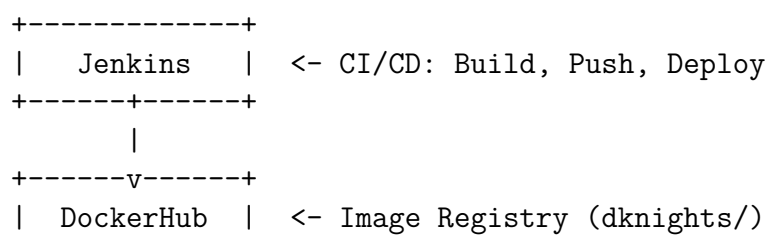
## 1.2 Tech Stack

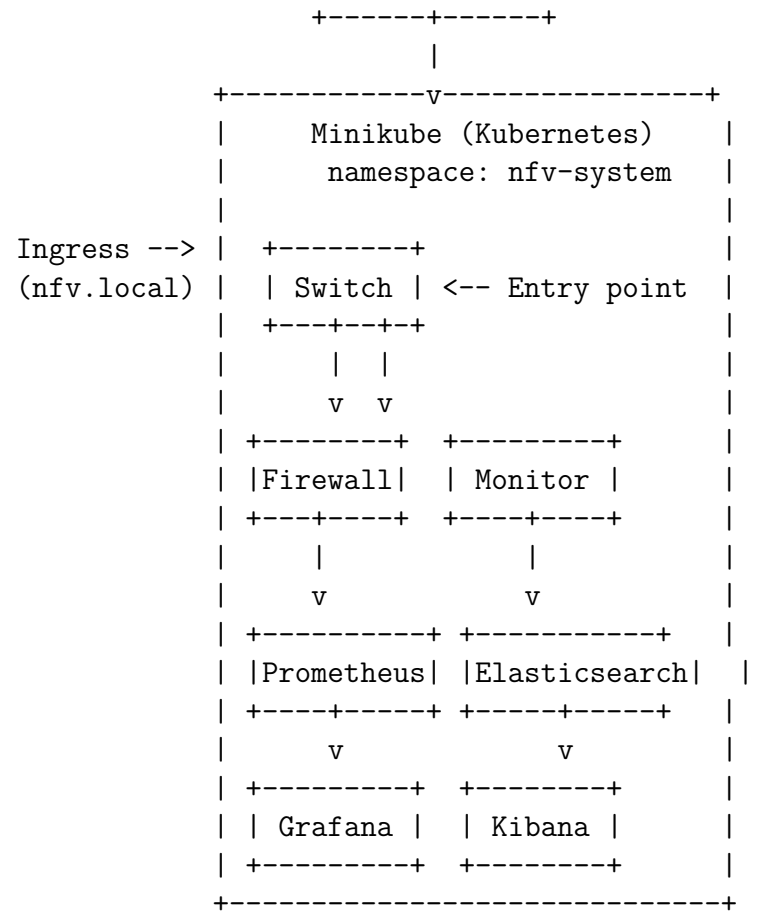
| Layer | Technology |
|---|---|
| Application | Python, Flask |
| Containerization | Docker |
| Orchestration | Kubernetes (Minikube) |
| CI/CD | Jenkins (Pipeline) |
| Image Registry | DockerHub (`dknights/`) |
| Metrics Monitoring | Prometheus |
| Metrics Visualization | Grafana |
| Log Storage | Elasticsearch |
| Log Visualization | Kibana |
| Ingress | NGINX Ingress Controller |
| Version Control | Git + GitHub |

Table 1: Technology Stack

# 2 Architecture

## 2.1 System Architecture Diagram






## 2.2 Component Roles

| Service | Port | Role |
|---|---|---|
| Switch | 5001 | Entry point. Routes traffic to Firewall and Monitor |
| Firewall | 5000 | Security layer. Blocks malicious traffic |
| Monitor | 5002 | Logging layer. Records all traffic to Elasticsearch |
| Prometheus | 9090 | Scrapes firewall metrics every 5 seconds |
| Grafana | 3000 | Visualizes metrics as dashboards |
| Elasticsearch | 9200 | Stores full traffic logs as searchable documents |
| Kibana | 5601 | Web UI to search and filter logs |

Table 2: Service Components

# 3 Microservices - Detailed Explanation

## 3.1 Switch Service (Port 5001)

**File:** `switch-service/app.py`

**Role:** The entry point of the entire system. Acts as a network switch/router.

**What it does:**

1. Receives incoming requests at `/route`
2. Forwards every request to the **Firewall** for security checking
3. Simultaneously sends the request to the **Monitor** for logging
4. Returns the firewall's decision (allowed/blocked) to the caller

Listing 1: Switch Service Core Logic

```python
@app.route("/route", methods=["POST"])
def route_request():
    # Send to Firewall for checking
    fw_response = requests.post(FIREWALL_URL, json=request.json)

    # Send to Monitor for logging
    requests.post(MONITOR_URL, json=request.json)

    # Return firewall's decision
    return jsonify(fw_response.json()), fw_response.status_code
```

## 3.2 Firewall Service (Port 5000)

**File:** `firewall-service/app.py`

**Role:** The security layer that inspects and filters traffic.

**What it does:**

1. Checks the source IP against a blocklist (`192.168.1.10`, `10.0.0.1`)
2. Scans request content for malicious keywords (`DROP`, `malicious`, `attack`)
3. Returns `403 Blocked` or `200 Allowed`
4. Exposes Prometheus metrics at `/metrics`

**Metrics exposed:**

- `allowed_requests_total` — Counter of allowed requests
- `blocked_requests_total` — Counter of blocked requests

Listing 2: Firewall Rules

```python
BLOCKED_IPS = ["192.168.1.10", "10.0.0.1"]
BLOCKED_KEYWORDS = ["DROP", "malicious", "attack"]

@app.route("/check", methods=["POST"])
def firewall_check():
    # Check blocked IPs
    if ip in BLOCKED_IPS:
        blocked_requests.inc()
        return {"status": "blocked", "reason": "IP blocked"}, 403

```

```
    # Check malicious content
    for keyword in BLOCKED_KEYWORDS:
        if keyword.lower() in content.lower():
            blocked_requests.inc()
            return {"status": "blocked", "reason": "malicious
               content"}, 403

    allowed_requests.inc()
    return {"status": "allowed"}, 200
```

### 3.3 Monitor Service (Port 5002)

**File:** `monitor-service/app.py`

**Role:** The logging/observability layer that records all traffic.

**What it does:**

1. Receives every request from the Switch (regardless of firewall decision)
2. Creates a log entry with IP, payload, and timestamp
3. Stores logs locally in memory (accessible via `/logs`)
4. Sends logs to Elasticsearch for persistent storage
5. Retries up to 3 times if Elasticsearch is unavailable

Listing 3: Monitor Logging Logic

```
ELASTIC_URL = "http://elasticsearch:9200/logs/_doc"

@app.route("/log", methods=["POST"])
def log_request():
    entry = {
        "ip": request.headers.get("X-Forwarded-For"),
        "data": request.json,
        "timestamp": datetime.now().isoformat()
    }
    logs.append(entry)  # Local storage
    requests.post(ELASTIC_URL, json=entry)  # Elasticsearch
    return {"status": "logged"}, 200
```

## 4 Monitoring vs Logging - Key Difference

**Prometheus + Grafana (Monitoring)**

**What:** Collects NUMBERS (metrics)
**Example:** "There were 50 blocked requests in the last hour"
**Use case:** Real-time dashboards, alerting, trend analysis
**Data format:** Time-series (metric name + value + timestamp)

**Elasticsearch + Kibana (Logging)**

**What:** Stores FULL LOGS (detailed records)
**Example:** "IP 192.168.1.10 sent payload {DROP table} at 3:42 PM"
**Use case:** Searching, filtering, forensics, auditing
**Data format:** JSON documents (IP, payload, timestamp)

**Summary:**

- Grafana answers: *"How much traffic?"* (graphs over time)
- Kibana answers: *"What was in the traffic?"* (searchable details)

# 5 CI/CD Pipeline (Jenkins)

## 5.1 Pipeline Stages

| # | Stage | What it does |
|---|---|---|
| 1 | Checkout | Clones code from GitHub (`dikshax86/NFV_SPE_MajorProject1`) |
| 2 | Build Images | Builds Docker images for all 3 services |
| 3 | Login to DockerHub | Authenticates using Jenkins credential `DockerHubCred` |
| 4 | Push Images | Pushes images to DockerHub (`dknights/`) |
| 5 | Deploy to K8s | Runs `deploy.sh` to apply all K8s manifests |

Table 3: Jenkins Pipeline Stages

## 5.2 Pipeline Flow Diagram

```
Developer pushes code to GitHub
         |
         v
Jenkins detects change (or manual Build Now)
         |
         v
[Checkout] -> Clone from GitHub
         |
         v
[Build Images] -> docker build (firewall, switch, monitor)
         |
         v
[Login to DockerHub] -> docker login using DockerHubCred
         |
         v
[Push Images] -> docker push dknights/firewall:v1
```

```
              -> docker push dknights/switch:v1
              -> docker push dknights/monitor:v1
         |
         v
[Deploy to K8s] -> kubectl apply all manifests
               -> kubectl rollout restart
```

## 5.3 Jenkinsfile

Listing 4: Jenkinsfile

```
pipeline {
    agent any
    environment {
        DOCKER_USER = "dknights"
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                url: 'https://github.com/dikshax86/
                   NFV_SPE_MajorProject1.git'
            }
        }
        stage('Build Images') {
            steps {
                sh '''
                docker build -t $DOCKER_USER/firewall:v1 ./
                   firewall-service
                docker build -t $DOCKER_USER/switch:v1 ./switch-
                   service
                docker build -t $DOCKER_USER/monitor:v1 ./monitor
                   -service
                '''
            }
        }
        stage('Login to DockerHub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'DockerHubCred',
                    usernameVariable: 'DOCKER_USERNAME',
                    passwordVariable: 'DOCKER_PASSWORD')]) {
                    sh 'echo $DOCKER_PASSWORD | docker login
                        -u $DOCKER_USERNAME --password-stdin'
                }
            }
        }
        stage('Push Images') {
            steps {
                sh '''
                docker push $DOCKER_USER/firewall:v1
```

```
                docker push $DOCKER_USER/switch:v1
                docker push $DOCKER_USER/monitor:v1
                '''
            }
        }
        stage('Deploy to Kubernetes') {
            steps {
                sh 'chmod +x deploy.sh'
                sh './deploy.sh'
            }
        }
    }
}
```

# 6 Kubernetes Deployment

## 6.1 Kubernetes Concepts Used

| Concept | Purpose in this project |
|---|---|
| Namespace | `nfv-system` isolates our project resources |
| Deployment | Manages pod replicas and rolling updates |
| Service (ClusterIP) | Internal DNS: `firewall:5000`, `switch:5001`, `monitor:5002` |
| Service (NodePort) | External access: Grafana (32210), Kibana (30602) |
| Ingress | Maps `nfv.local` hostname to Switch service |
| PersistentVolume | 1Gi disk for Elasticsearch data (survives restarts) |
| ConfigMap | Stores Prometheus scrape configuration |

Table 4: Kubernetes Concepts

## 6.2 Manifest Files

## 6.3 Deploy Script

Listing 5: deploy.sh

```
#!/bin/bash
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/storage.yaml
kubectl apply -f k8s/prometheus-config.yaml
kubectl apply -f k8s/prometheus.yaml
kubectl apply -f k8s/grafana.yaml
kubectl apply -f k8s/elasticsearch.yaml
kubectl apply -f k8s/kibana.yaml
kubectl apply -f k8s/firewall-deployment.yaml
kubectl apply -f k8s/monitor-deployment.yaml
kubectl apply -f k8s/switch-deployment.yaml
```

| File | Purpose |
|---|---|
| `namespace.yaml` | Creates `nfv-system` namespace |
| `storage.yaml` | PV + PVC for Elasticsearch (1Gi) |
| `firewall-deployment.yaml` | Firewall pod + ClusterIP service |
| `switch-deployment.yaml` | Switch pod + ClusterIP service |
| `monitor-deployment.yaml` | Monitor pod + ClusterIP service |
| `prometheus-config.yaml` | ConfigMap with scrape targets |
| `prometheus.yaml` | Prometheus deployment + service |
| `grafana.yaml` | Grafana deployment + NodePort service |
| `elasticsearch.yaml` | Elasticsearch deployment + service |
| `kibana.yaml` | Kibana deployment + NodePort service |
| `ingress.yaml` | NGINX ingress for `nfv.local` |

Table 5: Kubernetes Manifest Files

```
kubectl apply -f k8s/ingress.yaml

kubectl rollout restart deployment firewall -n nfv-system
kubectl rollout restart deployment monitor -n nfv-system
kubectl rollout restart deployment switch -n nfv-system
```

# 7 Request Flow (End to End)

```
1. User sends: POST http://nfv.local/route {"message": "hello"}
        |
        v
2. [NGINX Ingress Controller]
   - Matches host "nfv.local" and path "/"
   - Forwards to Switch service on port 5001
        |
        v
3. [Switch Service - port 5001]
   - Receives the request at /route
   - Sends to Firewall at http://firewall:5000/check
   - Sends to Monitor at http://monitor:5002/log
        |
        +-----> [Firewall Service - port 5000]
        |          - Checks IP against blocklist
        |          - Checks content for malicious keywords
        |          - Returns: allowed (200) or blocked (403)
        |          - Increments Prometheus counter
        |
        +-----> [Monitor Service - port 5002]
        |          - Creates log entry {ip, data, timestamp}
        |          - Stores in memory + sends to Elasticsearch
        |
```

```
        v
4. [Switch returns Firewall's response to user]
   - {"status": "allowed"} with 200
   - OR {"status": "blocked", "reason": "..."} with 403
```

# 8 Monitoring Flow

```
Every 5 seconds (background process):

    Prometheus ----scrapes----> http://firewall:5000/metrics
                                        |
                                        v
                                allowed_requests_total = 12
                                blocked_requests_total = 5
                                        |
                                Stored with timestamp
                                        |
                                        v
    Grafana ----queries----> Prometheus
                                        |
                                        v
                        Displays real-time graphs on dashboard
```

# 9 Logging Flow

```
On every request:

    Monitor Service ----POST----> Elasticsearch (port 9200)
                                          |
                                    Stores JSON document:
                                    {
                                      "ip": "192.168.49.1",
                                      "data": {"message": "hello"},
                                      "timestamp": "2026-05-15T15:39:51"
                                    }
                                          |
    Kibana ----reads----> Elasticsearch
                                          |
                                User searches/filters logs in UI
```

# 10 Complete Setup Guide

## 10.1 Prerequisites

- Docker installed and running
- Minikube installed

- kubectl installed
- Jenkins installed
- Java (for Jenkins)

## 10.2 Step 1: Start Minikube

```
minikube start
```

Starts a local single-node Kubernetes cluster.

## 10.3 Step 2: Enable Ingress Addon

```
minikube addons enable ingress
```

Installs the NGINX Ingress Controller so external traffic can reach services via `nfv.local`.

## 10.4 Step 3: Add nfv.local to /etc/hosts

```
echo "$(minikube ip) nfv.local" | sudo tee -a /etc/hosts
```

Maps the hostname `nfv.local` to Minikube's IP address.

## 10.5 Step 4: Login to DockerHub

```
docker login -u dknights
```

Enter your DockerHub password when prompted.

## 10.6 Step 5: Give Jenkins Access to Docker

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Adds jenkins user to docker group so it can build/push images.

## 10.7 Step 6: Give Jenkins Access to kubectl

```
# Copy kubeconfig
sudo mkdir -p /var/lib/jenkins/.kube
sudo cp ~/.kube/config /var/lib/jenkins/.kube/config
sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube

# Copy minikube certificates
sudo mkdir -p /var/lib/jenkins/.minikube/profiles/minikube
sudo cp ~/.minikube/ca.crt /var/lib/jenkins/.minikube/ca.crt
sudo cp ~/.minikube/profiles/minikube/client.crt \
    /var/lib/jenkins/.minikube/profiles/minikube/client.crt
```

```
sudo cp ~/.minikube/profiles/minikube/client.key \
    /var/lib/jenkins/.minikube/profiles/minikube/client.key
sudo chown -R jenkins:jenkins /var/lib/jenkins/.minikube

# Fix paths in kubeconfig
sudo sed -i \
    "s|/home/$(whoami)/.minikube|/var/lib/jenkins/.minikube|g" \
    /var/lib/jenkins/.kube/config
```

## 10.8 Step 7: Verify Jenkins kubectl Access

```
sudo -u jenkins kubectl get nodes
```

Expected output:

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   ...   v1.35.1
```

## 10.9 Step 8: Configure DockerHub Credentials in Jenkins UI

1. Open Jenkins: `http://localhost:8080`
2. Go to **Manage Jenkins** → **Credentials** → **System** → **Global credentials**
3. Click **Add Credentials**
4. Fill in:
   - Kind: `Username with password`
   - Username: `dknights`
   - Password: Your DockerHub password
   - ID: `DockerHubCred`
   - Description: `Docker Hub Credentials`
5. Click **Create**

## 10.10 Step 9: Create Jenkins Pipeline Job

1. In Jenkins → **New Item**
2. Name: `NFV`
3. Type: **Pipeline**
4. Under Pipeline section:
   - Definition: `Pipeline script from SCM`
   - SCM: `Git`
   - Repository URL: `https://github.com/dikshax86/NFV_SPE_MajorProject1.git`

- Branch: `*/main`
- Script Path: `Jenkinsfile`

5. Click **Save**

### 10.11 Step 10: Run the Pipeline

Click **Build Now** on the pipeline job in Jenkins.

### 10.12 Step 11: Verify Deployment

```
kubectl get all -n nfv-system
```

All pods should show `STATUS: Running`.

## 11 Testing Guide

### 11.1 Test 1: Allowed Request

```
curl -X POST http://nfv.local/route \
  -H "Content-Type: application/json" \
  -d '{"message": "hello world"}'
```

**Expected response:**

```
{"status": "allowed"}
```

### 11.2 Test 2: Blocked Request (Malicious Keyword)

```
curl -X POST http://nfv.local/route \
  -H "Content-Type: application/json" \
  -d '{"message": "malicious attack"}'
```

**Expected response:**

```
{"status": "blocked", "reason": "malicious content"}
```

### 11.3 Test 3: Blocked Request (Blocked IP)

```
curl -X POST http://nfv.local/route \
  -H "Content-Type: application/json" \
  -H "X-Forwarded-For: 192.168.1.10" \
  -d '{"message": "hello"}'
```

**Expected response:**

```
{"status": "blocked", "reason": "IP blocked"}
```

### 11.4 Test 4: Generate Bulk Traffic

```
# 10 allowed requests
for i in {1..10}; do
  curl -s -X POST http://nfv.local/route \
    -H "Content-Type: application/json" \
    -d '{"message": "hello"}'
done

# 5 blocked requests
for i in {1..5}; do
  curl -s -X POST http://nfv.local/route \
    -H "Content-Type: application/json" \
    -d '{"message": "malicious attack"}'
done
```

### 11.5 Test 5: Check Monitor Logs

```
kubectl exec -n nfv-system deployment/monitor -- \
  python -c "import requests; \
  r = requests.get('http://localhost:5002/logs'); \
  print(r.json())"
```

### 11.6 Test 6: Check Elasticsearch Logs

```
kubectl exec -n nfv-system deployment/elasticsearch -- \
  curl -s http://localhost:9200/logs/_search?pretty
```

### 11.7 Test 7: Check Prometheus Metrics

```
kubectl exec -n nfv-system deployment/prometheus -- \
  wget -qO- "http://localhost:9090/api/v1/query?query=
    allowed_requests_total"
```

## 12 Accessing the UIs

### 12.1 Grafana (Metrics Dashboards)

- **URL:** `http://<minikube-ip>:32210`
- **Login:** admin / admin
- **Purpose:** Real-time graphs of firewall traffic

**Setup Grafana Dashboard:**

1. Login with admin/admin

2. Go to Connections → Data sources → Add Prometheus
3. URL: `http://prometheus:9090`
4. Save & Test
5. Create Dashboard → Add Panel
6. Metric: `allowed_requests_total`
7. Add Query B: `blocked_requests_total`
8. Save dashboard

## 12.2 Kibana (Log Search)

- **URL:** `http://<minikube-ip>:30602`
- **Purpose:** Search and filter traffic logs

**Setup Kibana:**

1. Open Kibana URL
2. Go to Management → Data Views
3. Create index pattern: `logs*`
4. Time field: `timestamp`
5. Go to Discover to search logs

## 12.3 Prometheus (Raw Metrics)

```
kubectl port-forward svc/prometheus 9090:9090 -n nfv-system
```

Then open: `http://localhost:9090`
Type a metric name and click Execute:

- `allowed_requests_total`
- `blocked_requests_total`
- `rate(allowed_requests_total[1m])`

| Image | Source | Description |
|---|---|---|
| `dknights/firewall:v1` | `firewall-service/` | Python Flask firewall |
| `dknights/switch:v1` | `switch-service/` | Python Flask router |
| `dknights/monitor:v1` | `monitor-service/` | Python Flask logger |
| `grafana/grafana` | Official | Visualization |
| `prom/prometheus` | Official | Metrics collector |
| `elasticsearch:8.13.0` | Elastic official | Search engine |
| `kibana:8.13.0` | Elastic official | Log UI |

Table 6: Docker Images Used

| Problem | Solution |
|---|---|
| Pods in ImagePullBackOff | Images not on DockerHub. Run Jenkins pipeline to build & push. |
| Pods in ContainerCreating | Minikube is pulling images. Wait 1-2 minutes. |
| Pods in Terminating | Normal after rollout restart. Old pods shutting down. |
| Jenkins Deploy fails | Copy fresh `~/.kube/config` and minikube certs to Jenkins. |
| Grafana shows No data | Configure Prometheus data source: `http://prometheus:9090` |
| Elasticsearch empty | Send test requests first. Check monitor pod logs. |

Table 7: Common Issues

# 13 Docker Images

# 14 Troubleshooting

# 15 Useful Commands

```
# Check all resources
kubectl get all -n nfv-system

# Check pod logs
kubectl logs deployment/firewall -n nfv-system
kubectl logs deployment/switch -n nfv-system
kubectl logs deployment/monitor -n nfv-system

# Describe pod (for debugging)
kubectl describe pod <pod-name> -n nfv-system

# Port-forward Prometheus
kubectl port-forward svc/prometheus 9090:9090 -n nfv-system

# Restart a deployment
kubectl rollout restart deployment firewall -n nfv-system
```

```
# Delete everything and start fresh
kubectl delete namespace nfv-system

# Check minikube IP
minikube ip

# Check ingress
kubectl get ingress -n nfv-system
```

# 16 Project File Structure

```
NFV_Diksha/
|-- Jenkinsfile                    # CI/CD pipeline definition
|-- deploy.sh                      # K8s deployment script
|-- docker-compose.yml             # Local development setup
|-- post.json                      # Sample test payload
|
|-- switch-service/                # Entry point microservice
|   |-- app.py                     # Flask app (routes traffic)
|   |-- Dockerfile                 # Container definition
|   +-- requirements.txt           # Python dependencies
|
|-- firewall-service/              # Security microservice
|   |-- app.py                     # Flask app (blocks/allows)
|   |-- Dockerfile                 # Container definition
|   +-- requirements.txt           # Python dependencies
|
|-- monitor-service/               # Logging microservice
|   |-- app.py                     # Flask app (logs to ES)
|   |-- Dockerfile                 # Container definition
|   +-- requirements.txt           # Python dependencies
|
+-- k8s/                           # Kubernetes manifests
    |-- namespace.yaml
    |-- storage.yaml
    |-- firewall-deployment.yaml
    |-- switch-deployment.yaml
    |-- monitor-deployment.yaml
    |-- ingress.yaml
    |-- prometheus-config.yaml
    |-- prometheus.yaml
    |-- grafana.yaml
    |-- elasticsearch.yaml
    +-- kibana.yaml
```